

El proyecto, en una frase

Construimos una plataforma ficticia de comida a domicilio que envía sus acciones a ClickHouse y las representa en un dashboard.

Se llama Delivery Atlas. Está inspirada en el recorrido de Uber Eats o Glovo, pero el negocio, los restaurantes y los pedidos de nuestra app serán inventados. El objetivo de BD2 es demostrar una base de datos analítica con mucho volumen y explicar cómo funciona.

Qué verá el profesor

Una persona busca un restaurante, abre un producto, añade al carrito y confirma un pedido simulado. Cada acción deja un registro. Una API recibe esos registros; ClickHouse los guarda y el dashboard muestra recuentos por fecha, ciudad y acción.

Qué existe hoy

Implementado: SQL, API, cola persistente, publicación por lotes, consultas, generador y pruebas. Un millón de eventos ficticios ya está comprobado en la nueva base. También conservamos el laboratorio con datasets públicos y sus paneles.

Por construir: miniapp propia, dashboard específico y prueba completa en el ordenador de otro compañero. La raíz del puerto 8001 muestra información de la API, no una aplicación terminada.

Persona	Se encarga de	Entrega
Jere + Rayo	Base e ingesta	Infraestructura y mediciones
Anuar	Miniapp	Acciones que generan eventos
Echenique	Dashboard y demo	Visualización y reproducción

Primera reunión: leer esta página juntos, repartir los archivos y arrancar la ruta pequeña de la página 7. Nadie necesita descargar cientos de millones de filas para comenzar.

Qué datos tenemos y qué significan

Hay dos partes: un laboratorio con fuentes públicas y una plataforma propia con eventos ficticios o acciones manuales.

Fuente	Volumen cargado	Qué representa
OTTO	216.716.096	Clics, carritos y pedidos
REES46	411.709.736	Vistas, carritos y compras
Ele.me	2.170.299	Muestras de recomendación
Glovo	2.887.444	Productos del catálogo
Atlas: generador	1.000.000	Acciones inventadas

OTTO y REES46 aportan volumen de comercio electrónico. Ele.me acerca el caso al reparto de comida en China: cargamos una parte del dataset, no todos los datos anunciados. Cada fila es una muestra, no necesariamente un clic ni una sesión completa. Glovo aporta productos; un producto no es un clic.

La nueva base: delivery_atlas

synthetic: eventos creados por el generador. Hoy: tres ciudades y cinco pasos por sesión, siempre completos. Sirven para probar carga, no para estimar demanda o conversión.

demo_live: acciones nuevas que haremos manualmente en la miniapp ficticia. Se registran cuando alguien usa la demo; no pertenecen a clientes reales de Uber Eats.

Cómo lo presentamos con rigor

Separar fuentes y mostrar su etiqueta de origen. La reproducción de un archivo antiguo inserta datos ahora, pero no cambia la fecha original del evento. El laboratorio tiene además 1.083.580.480 filas derivadas de OTTO para estrés: no son clics nuevos.

No mezclamos todos los números como si describieran una población. El millón Atlas está verificado por su identificador de ejecución. Las filas de comprobación añadidas después se contabilizan aparte.

Recuentos históricos comprobados de nuevo el 7/10/2026. Fuentes y significado: README.md; resultados propios: docs/RESULTADOS-ATLAS.md.

Cómo viaja un evento

Una acción se recibe, se conserva en cola y después se publica en ClickHouse. El dashboard consulta ClickHouse a través de la API.

Miniapp de Anuar o generador de Jere
API Python: valida y recibe el evento
Cola SQLite: conserva lo pendiente
Publicador: inserta lotes en ClickHouse
Detalle + agregado diario: delivery_atlas
API de lectura: dashboard de Echenique

Dos servicios, un nodo de base de datos

Un nodo es una instancia de ClickHouse. Tenemos uno. La API es otro contenedor, pero no es un segundo nodo de ClickHouse. Docker Compose arranca ambos; los volúmenes conservan la base y la cola al reiniciar.

API local: **127.0.0.1:8001**. ClickHouse local: **127.0.0.1:8124**. Dentro de Docker se comunican por clickhouse:8123. Los paneles históricos usan el puerto 8000 y otra API.

Las tablas, sin complicarlo

app_events: guarda cada acción con ID, sesión, hora, ciudad, tipo y origen. Particiona por mes y ordena los datos para las consultas. El orden físico no es una restricción de unicidad.

app_daily_rollup_mv: procesa nuevas inserciones. **app_daily_rollup**: conserva conjuntos de IDs por grupo. **app_daily_counts**: cuenta IDs distintos. Así un reenvío no suma dos veces la misma acción en ese grupo.

SQL aplicado: sql/atlas.sql. ReplacingMergeTree elimina copias al fusionar; FINAL las resuelve al leer. AggregatingMergeTree combina estados. Un volumen no equivale a una réplica ni a una copia de seguridad.

Jere: base, ingesta y pruebas

Tu parte, con Rayo: que los eventos lleguen correctamente a la base y que podamos demostrar qué ocurre con carga y fallos.

Lo que ya hemos implementado

1. **SQL:** base delivery_atlas, detalle, agregado y vista de lectura. Archivo: sql/atlas.sql; consultas explicadas: sql/atlas_consultas.sql.
2. **Backend:** contrato y validación, cola persistente, publicación por lotes y endpoints. Archivos: atlas/contract.py, store.py, clickhouse.py y server.py.
3. **Entorno:** Dockerfile.atlas y perfil atlas de compose.yaml. La cola está en un volumen propio; la base conserva su volumen existente.
4. **Evidencia:** generador, benchmark, prueba de reintentos y recuperación. Scripts: atlas.py, benchmark_atlas.py, check_atlas.py y recover_atlas.py, dentro de scripts/.

Lo siguiente que tienes que hacer

5. Resolver dudas del contrato con Anuar y Echenique. Revisar un recorrido completo y comprobar el ID de cada acción en ClickHouse.
6. Medir el tiempo desde el clic hasta el dashboard y el crecimiento del disco. Mejorar las sesiones con abandonos y horarios sin presentarlas como comportamiento real.
7. Investigar el coste del agregado exacto: hoy no gana al detalle en nuestra medición. Optimizar solo si mantiene los mismos recuentos al repetir eventos.
8. Documentar copia/restauración. Después decidir si aporta valor un segundo nodo; por ahora la demo funciona con uno.

Tu entrega se acepta cuando...

El entorno arranca, el evento llega, reintentar no infla el recuento lógico y cada afirmación tiene una prueba. El límite de la cola y los costes observados aparecen en la documentación.

Anuar: miniapp y acciones

Tu parte: construir una pequeña app que podamos usar delante del profesor y que genere los eventos correctos.

Archivos que creas

web/delivery.html, **web/delivery.js** y **web/delivery.css**. No hay que hacer login, cobros ni asignación de repartidores. Usa un catálogo ficticio pequeño con IDs estables.

Pasos, en este orden

1. Permitir elegir ciudad y buscar. Después abrir restaurante, abrir producto, añadir al carrito y confirmar un pedido simulado.
2. Al comenzar, crear **session_id** con `crypto.randomUUID()`. Cada acción crea su propio **event_id** y su fecha con `new Date().toISOString()`.
3. Enviar un objeto `{"events": [...]}` a **POST /api/events**. Usar `source=delivery-miniapp` y `source_kind=demo_live`. Ejemplo completo de campos: docs/INFRAESTRUCTURA-ATLAS.md.
4. Acciones básicas: `search`, `restaurant_opened`, `product_opened`, `cart_added` y `order_submitted`. Abrir restaurante requiere `restaurant_id`; producto y carrito requieren además `product_id`. Añadir ciudad y sección siempre.
5. Guardar el objeto antes de enviarlo. Si la red falla, reenviar **el mismo ID, fecha y contenido**, sin generar otro evento. Coordinar la espera y los errores con Jere.

Qué significa la respuesta

202: guardado en cola, pendiente de publicación. **200**: ese evento ya se había aceptado.
400/409/413: corregir formato, conflicto o tamaño. **503**: esperar y reintentar el objeto original.
Mostrar el estado sin afirmar que ya está en ClickHouse.

Tu entrega se acepta cuando...

Echenique realiza un recorrido y encuentra sus acciones en la base. Repetir un envío no suma una nueva acción. La página funciona en el puerto 8001 después de reconstruir el contenedor.

Echenique: dashboard y reproducción

Tu parte: enseñar los datos que realmente guarda ClickHouse y comprobar que otra persona puede repetir la demo.

Archivos que creas

`web/atlas-dashboard.html`, `web/atlas-dashboard.js`, `web/atlas-dashboard.css`, `docs/DEMO-ATLAS.md` y `docs/PRUEBAS-EQUIPO.md`.

Pasos, en este orden

1. **GET /api/summary:** mostrar eventos por día, ciudad y acción. Ofrecer filtros `from`, `to`, `city_id`, `source_kind` y `event_type`. Respetar `truncated` si faltan grupos en la respuesta.
2. **GET /api/recent:** mostrar los últimos 30 eventos globales con ID, sesión y horas. Este endpoint no filtra por ciudad; etiquetar el listado como `global`.
3. **GET /api/status:** enseñar pendientes y publicados por la cola principal. Estos contadores no son el total de todas las inserciones posibles en ClickHouse.
4. Refrescar cada dos segundos sin solapar peticiones. Indicar última actualización, carga y errores. Diferenciar `synthetic` y `demo_live`. El millón simulado no es actividad de usuarios actuales.
5. Arrancar en tu ordenador con la página 7. Anotar sistema, versiones, comandos, resultado y cualquier arreglo necesario. Ejecutar el recorrido de Anuar.
6. Preparar una demo breve: generar, ver el panel, consultar detalle y agregado, repetir un evento y mostrar CRUD en una tabla de ensayo.

Tu entrega se acepta cuando...

El panel lee la API y una acción aparece tras publicarse. La guía permite reproducirlo sin los datasets gigantes. No llamar conversión a pedidos/clics: necesitaría una definición y sesiones con abandonos.

CRUD ya disponible: `python3 scripts/crud_demo.py`. Opera en `delivery.demo_events`; no modifica el flujo analítico Atlas. Acuerda con Jere el momento de ejecutarlo.

Arrancar y comprobar la ruta pequeña

Desde la carpeta del repositorio: Docker/Compose y Python 3.10 o posterior. No hace falta descargar los archivos históricos.

1. Construir y arrancar

```
docker compose -f compose.yaml --profile atlas up -d --build --wait
docker compose -f compose.yaml --profile atlas ps
```

El primer arranque descarga las imágenes y aplica el SQL. Deben aparecer los dos servicios healthy. Para recibir las actualizaciones de Python, SQL o web, repetir el comando con --build.

2. Verificar contrato y reintentos

```
python3 -m unittest discover -s tests -v
python3 scripts/check_atlas.py
```

Las pruebas deben pasar. El script comprueba HTTP, conflicto de ID, contrato inválido y reenvío tras una respuesta perdida. Ejecutarlo sin carga simultánea.

3. Crear una carga pequeña

```
python3 scripts/atlas.py generate --events 10000 --batch 1000 --seed 42
python3 scripts/atlas.py status
```

Esperar a **pending=0**. Repetir el comando status si aún quedan eventos. Consultar en el navegador **<http://127.0.0.1:8001/api/summary>** y **[/api/recent](http://127.0.0.1:8001/api/recent)**. Al principio veréis JSON; las interfaces son la siguiente entrega.

4. Integrar los archivos web

Cuando Anuar y Echenique los creen, reconstruir y abrir **[/delivery.html](#)** y **[/atlas-dashboard.html](#)** en el puerto 8001. Se sirven junto a la API para evitar una configuración adicional de CORS.

Para parar: docker compose -f compose.yaml --profile atlas stop. Conservar los volúmenes; down -v los borra. El puerto 8000 pertenece al laboratorio anterior y no es la nueva API.

El script que crea este PDF requiere reportlab y fuentes Liberation Sans. Son herramientas opcionales de documentación; la demo Python no necesita instalarlas.

Resultados y próxima entrega

Podemos enseñar una base funcionando y un millón de eventos ficticios verificados. Todavía falta conectar las dos interfaces.

Comprobación	Resultado local del 7/10/2026
Carga Atlas	1.000.000 eventos verificados en ClickHouse
Envío a la cola	323,59 s; aprox. 3090 eventos/s
Reintento / conflicto	Sin nuevo evento lógico / HTTP 409
Reinicio con pendiente	Se recuperó; cuenta lógica = 1
Detalle / agregado	Mismo resultado: 16 grupos

La velocidad de envío incluye generar, validar y confirmar la cola; no mide por sí sola la rapidez de ClickHouse. Se observó un fallo transitorio de publicación durante la carga y se recuperó. La prueba de reinicio usa paradas ordenadas, no pérdida de disco.

Un resultado que debemos explicar

El agregado exacto aún no fue más rápido: mediana 0,703 s frente a 0,498 s del detalle, en tres ejecuciones por alternativa con caché SQL desactivada. Los conjuntos de IDs protegen contra reenvíos, pero combinarlos tiene coste. La mejora anterior de OTTO corresponde a otra consulta y otro esquema.

Orden de la siguiente entrega

Primero: Anuar y Echenique construyen sus pantallas con el contrato disponible. **Después:** Echenique prueba el recorrido; Jere revisa IDs, recuentos y retraso. **Finalmente:** ejecutar en otro ordenador, registrar condiciones y ensayar juntos.

Reparto de la exposición final

Propuesta de 30 minutos: Jere, 8 min de ClickHouse e infraestructura; Anuar, 5 min de miniapp y evento; Echenique, 5 min de dashboard y reproducción; Jere, 2 min de pruebas y límites; 10 min para discusión y preguntas. Confirmar el formato de 20 + 10 con el profesor.

Guía técnica: docs/INFRAESTRUCTURA-ATLAS.md. Reparto editable: docs/PLAN-EQUIPO.md. Evidencia: docs/RESULTADOS-ATLAS.md y docs/evidencia-atlas.json. Referencias oficiales de motores enlazadas en la guía técnica.